JDBC Drivers

JDBC drivers implement the defined interfaces in the JDBC API, for interacting with your database server.

For example, using JDBC drivers enable you to open database connections and to interact with it by sending SQL or database commands then receiving results with Java.

The *Java.sql* package that ships with JDK, contains various classes with their behaviours defined and their actual implementaions are done in third-party drivers. Third party vendors implements the *java.sql.Driver* interface in their database driver.

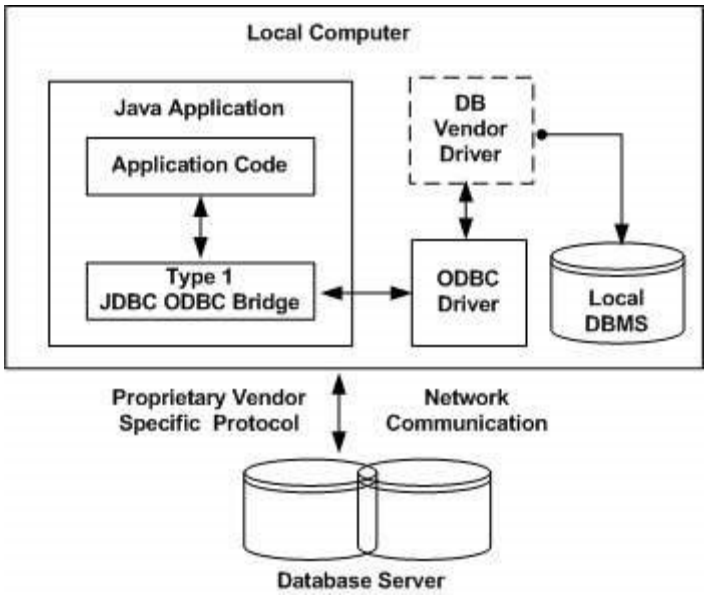
JDBC Drivers Types

JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates. Sun has divided the implementation types into four categories, Types 1, 2, 3, and 4, which is explained below −

Type 1: JDBC-ODBC Bridge Driver

In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine. Using ODBC, requires configuring on your system a Data Source Name (DSN) that represents the target database.

When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.



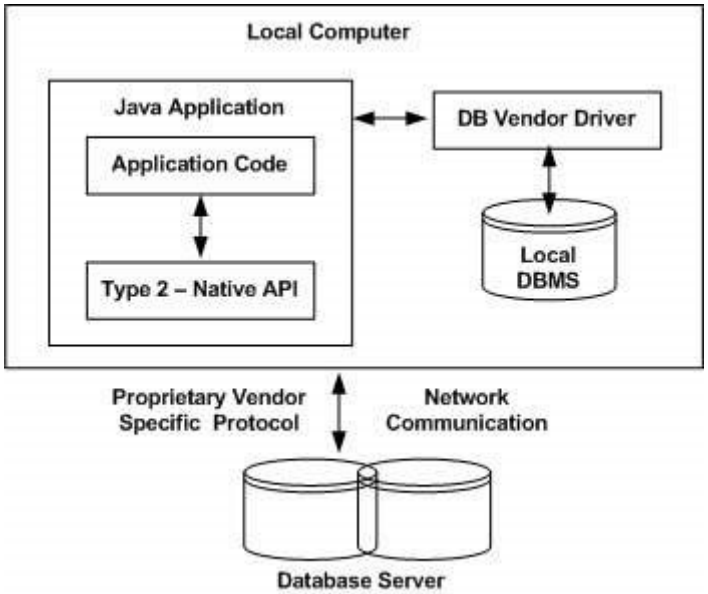
The JDBC-ODBC Bridge that comes with JDK 1.2 is a good example of this kind of driver.

Type 2: JDBC-Native API

In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls, which are unique to the database. These drivers are typically provided by the database

vendors and used in the same manner as the JDBC-ODBC Bridge. The vendor-specific driver must be installed on each client machine.

If we change the Database, we have to change the native API, as it is specific to a database and they are mostly obsolete now, but you may realize some speed increase with a Type 2 driver, because it eliminates ODBC's overhead.

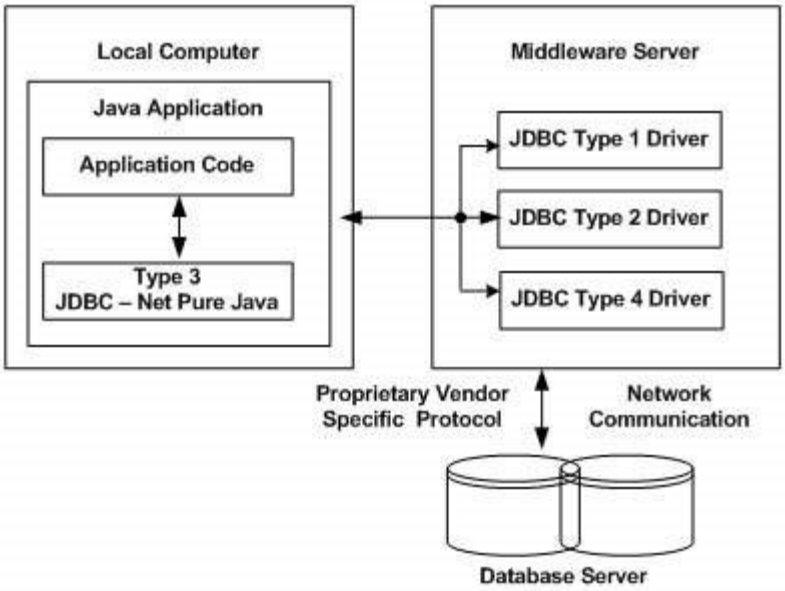


The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

Type 3: JDBC-Net pure Java

In a Type 3 driver, a three-tier approach is used to access databases. The JDBC clients use standard network sockets to communicate with a middleware application server. The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server.

This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.



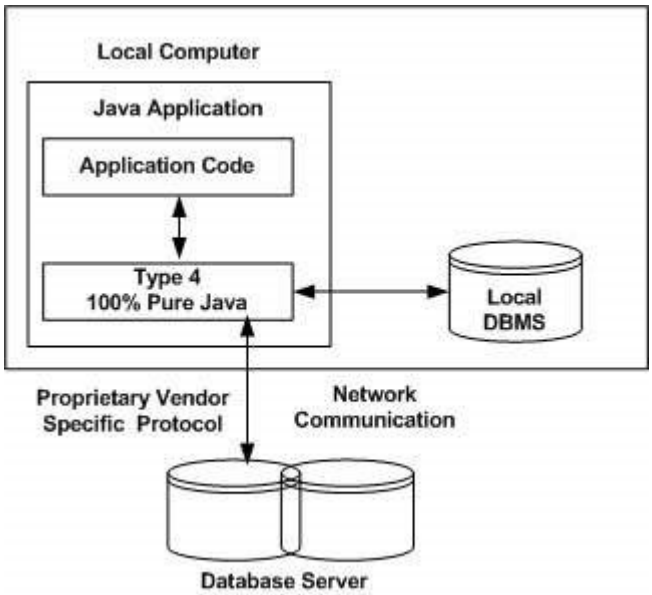
You can think of the application server as a JDBC "proxy," meaning that it makes calls for the client application. As a result, you need some knowledge of the application server's configuration in order to effectively use this driver type.

Your application server might use a Type 1, 2, or 4 driver to communicate with the database, understanding the nuances will prove helpful.

Type 4: 100% Pure Java

In a Type 4 driver, a pure Java-based driver communicates directly with the vendor's database through socket connection. This is the highest performance driver available for the database and is usually provided by the vendor itself.

This kind of driver is extremely flexible, you don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically.



MySQL's Connector/J driver is a Type 4 driver. Because of the proprietary nature of their network protocols, database vendors usually supply type 4 drivers.

Merging Data from Multiple Tables: Joining, Manipulating:

```
import java.sql.*;

 public class jt {

 public static void main(String a[])

 {

Connection con = null;

String url = "jdbc:mysql://localhost:3306/";

String dbName = "college";
```

```
String driver = "com.mysql.cj.jdbc.Driver";
String userName = "root";
String password = "root";
try {
 Class.forName("com.mysql.cj.jdbc.Driver");
 con = DriverManager.getConnection(url+dbName,userName,password);
 Statement st = con.createStatement();
 ResultSet rs = st.executeQuery("select * from Stu s,dept d where s.sid=d.sid");
 while( rs.next() ) {
 String data = rs.getString(1);

 System.out.println( data );
 }
 System.out.println("Results");
 st.close();
 }
 catch( Exception e ) {
 System.out.println(e.getMessage());
 e.printStackTrace();
 }
 }
}
```

Statement interface

The **Statement interface** provides methods to execute queries with the database. The statement interface is a factory of ResultSet i.e. it provides factory method to get the object of ResultSet.

Commonly used methods of Statement interface:

The important methods of Statement interface are as follows:

1) public ResultSet executeQuery(String sql): is used to execute SELECT query. It returns the object of ResultSet.

2) public int executeUpdate(String sql): is used to execute specified query, it may be create, drop, insert, update, delete etc.

3) public boolean execute(String sql): is used to execute queries that may return multiple results.

4) public int[] executeBatch(): is used to execute batch of commands.

PreparedStatement interface

The PreparedStatement interface is a subinterface of Statement. It is used to execute parameterized query.

Let's see the example of parameterized query:

```
1. String sql="insert into emp values(?,?,?)";
```

As you can see, we are passing parameter (?) for the values. Its value will be set by calling the setter methods of PreparedStatement.

Why use PreparedStatement?

Improves performance: The performance of the application will be faster if you use PreparedStatement interface because query is compiled only once.

How to get the instance of PreparedStatement?

The prepareStatement() method of Connection interface is used to return the object of PreparedStatement. Syntax:

```
1. public PreparedStatement prepareStatement(String query)throws SQLException{}
```

Methods of PreparedStatement interface

The important methods of PreparedStatement interface are given below:

Method	Description
public void setInt(int paramIndex, int value)	sets the integer value to the given parameter index.
public void setString(int paramIndex, String value)	sets the String value to the given parameter index.
public void setFloat(int paramIndex, float value)	sets the float value to the given parameter index.
public void setDouble(int paramIndex, double value)	sets the double value to the given parameter index.
public int executeUpdate()	executes the query. It is used for create, drop, insert, update, delete etc.
public ResultSet executeQuery()	executes the select query. It returns an instance of ResultSet.

➢ Example of PreparedStatement interface that inserts the record

First of all create table as given below:

1. Create table Students(id int(10), name varchar(20), age int(2), branch varchar())Now insert records in this table by the code given below:

```
import java.sql.*;

public class insert {

public static void main(String arg[]){

try{

 Class.forName("com.mysql.cj.jdbc.Driver");

 try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college","root",
"root")) {

 PreparedStatement stmt=con.prepareStatement("insert into Students(id, name, age, branch)" +
"values(?,?,?,?)");
```

```
            stmt.setInt(1,7);
            stmt.setString(2,"Amrit");
            stmt.setInt(3,21);
            stmt.setString(4,"IT");
            int i=stmt.executeUpdate();
            System.out.println("Data is saved");
        }
        }
        catch (ClassNotFoundException | SQLException e){
            }
    }
    }
```

Output:

```
SQL 1 execution ×   SQL 2 execution ×   SQL 3 execution ×   SQL 4 execution ×   JavaA
run:
Data is saved
BUILD SUCCESSFUL (total time: 0 seconds)
```

➢ Example of PreparedStatement interface that updates the record

```
PreparedStatement stmt=con.prepareStatement("update emp set name=? where id=?");
stmt.setString(1,"Sonoo");//1 specifies the first parameter in the query i.e. name
stmt.setInt(2,101);

int i=stmt.executeUpdate();
System.out.println(i+" records updated");
```

➢ Example of PreparedStatement interface that deletes the record

```
PreparedStatement stmt=con.prepareStatement("delete from emp where id=?");
stmt.setInt(1,101);

int i=stmt.executeUpdate();
System.out.println(i+" records deleted");
```

- **Example of PreparedStatement interface that retrieve the records of a table**

```
PreparedStatement stmt=con.prepareStatement("select * from emp");
ResultSet rs=stmt.executeQuery();
while(rs.next()){
System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

Transaction Processing:

Transaction represents **a single unit of work**.

The ACID properties describes the transaction management well. ACID stands for Atomicity, Consistency, isolation and durability.

Atomicity means either all successful or none.

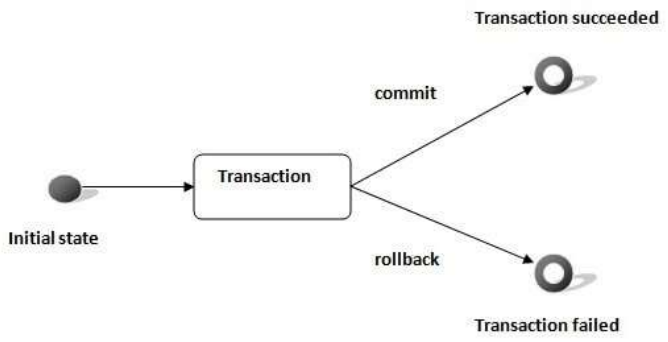
Consistency ensures bringing the database from one consistent state to another consistent state.

Isolation ensures that transaction is isolated from other transaction.

Durability means once a transaction has been committed, it will remain so, even in the event of errors, power loss etc.

Advantage of Transaction Mangaement

fast performance It makes the performance fast because database is hit at the time of commit.



Method	Description

void setAutoCommit(boolean status)	It is true bydefault means each transaction is committed bydefault.
void commit()	commits the transaction.
void rollback()	cancels the transaction.

In JDBC, **Connection interface** provides methods to manage transaction.

Transaction management using PreparedStatement.

```
import java.sql.*;

import java.io.*;

class TMang{

public static void main(String args[]){

try{

Class.forName("com.mysql.cj.jdbc.Driver");

Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college","root",
"root");

con.setAutoCommit(false);

PreparedStatement ps=con.prepareStatement("insert into emp values(?,?,?)");

BufferedReader br=new BufferedReader(new InputStreamReader(System.in));

while(true){

 System.out.println("enter id");

String s1=br.readLine();

int id=Integer.parseInt(s1);

System.out.println("enter name");

String name=br.readLine();

System.out.println("enter salary");

String s3=br.readLine();

int salary=Integer.parseInt(s3);

ps.setInt(1,id);

ps.setString(2,name);

ps.setInt(3,salary);
```

```
ps.executeUpdate();
System.out.println("commit/rollback");
String answer=br.readLine();
if(answer.equals("commit")){
con.commit();
}
if(answer.equals("rollback")){
con.rollback();
}
System.out.println("Want to add more records y/n");
String ans=br.readLine();
if(ans.equals("n")){
break;
}
}
con.commit();
System.out.println("record successfully saved");
 con.close();//before closing connection commit() is called
}catch(Exception e){System.out.println(e);}
}}
```

Output:

```
enter id
15
enter name
Deepali
enter salary
40000
commit/rollback
commit
Want to add more records y/n
n
record successfully saved
BUILD SUCCESSFUL (total time: 27 seconds)
```